ChatGPT

Everabot: A Self-Evolving AI Agent Architecture (White Paper Outline)

Vision and Goals

Everabot is conceived as a **single autonomous agent** you can instruct in plain English to **"learn any topic"** or **"build any project"** – and it will **continually improve itself** to fulfill those tasks. The agent can **learn new skills without forgetting old ones**, **optimize its own code and performance**, and **prove what it learned** through tests and logs. Crucially, Everabot combines **context evolution** (Agentic Context Engineering) with **adaptive models**: it primarily improves via contextual strategies [1], but it can also **update its own weights or create new models** when beneficial. This yields a system that becomes more capable over time with minimal human intervention, running entirely on local resources (e.g. a Docker container). The ultimate vision is an AI that **"does anything and everything"** you ask – whether solving math problems, building a website or game, parsing files, or even refactoring its own UI – all while **preserving prior knowledge** and working within given rules.

Core Behaviors and Constraints

To realize this vision safely and reliably, Everabot follows a strict behavior contract:

- **Two Operating Modes:** It has a **Mission mode** (focus on the user's requested task, e.g. *"Build a game I can sell"*) and a **Self-Improve mode** (focus only on improving its own internals – knowledge, code efficiency, interface – without advancing a user mission). The agent will clearly distinguish these modes to avoid self-modifications when it's supposed to be delivering a product.
- **Rule Obedience:** Everabot loads an **overarching ruleset** (in `ruleset/`) that defines hard constraints (safety, ethical limits, user's boundaries). All actions and code changes are validated against these rules. It will have built-in checks and tests to **ensure it never violates base rules** or attempts disallowed behaviors.
- **No Forgetting Prior Skills:** The agent maintains a growing **library of skills** (methods, prompts, tools) and a suite of **regression tests** for each skill domain. When it learns something new, it **re-runs tests for older skills** to confirm it hasn't regressed. If any old capability fails its test, the new change is rolled back. This addresses the *catastrophic forgetting* problem – every learning loop must prove it preserved prior abilities before its results are "promoted" as the new state.
- **Evidence and Explainability:** Every time Everabot learns or self-modifies, it produces **proof logs** with traceability: sources researched, distilled notes or strategies, test results, and a summary of changes. This provides a form of **explainable AI** – you can inspect why it made each decision. For example, if it learned web development, it would log which tutorials or docs it read and how it validated the resulting website.
- **Efficiency Bias:** Given it runs locally with limited resources, the agent is biased toward **efficient solutions**. It minimizes prompt size and memory usage via advanced prompt compression (e.g. the LLMLingua method can compress prompts by up to 20× with minimal performance loss [2]), uses a compact memory store, and prefers algorithmic or compiled solutions when possible. Everabot

treats **speed and RAM as key metrics** to optimize – it will prefer a strategy that is faster or more memory-efficient if it achieves the same goal.

- **Continuous Validation:** A multi-objective criteria is enforced for any change: *new capabilities must improve or meet a target score, and all previously met criteria must remain at least as good as before*. Each learning cycle ends with a "promotion gate" where the agent's challenger version (with new changes) is evaluated against the current champion version. Only if the challenger **wins on new skills without any regressions** is it accepted as the new champion state. Otherwise, it auto-rollback to the previous stable state. This **champion/challenger testing** ensures only net-positive changes persist.

Learning New Skills: The "Learn X" Pipeline

When you command Everabot to **"Learn X"**, it initiates a self-directed curriculum for that topic. This process uses a loop of **research → distillation → practice → integration**, drawing on recent advances in *Agentic Context Engineering (ACE)* [1] and *Retrieval-Augmented Generation (RAG)* [3]:

1. **Research Phase (Knowledge Acquisition with RAG):** The agent first gathers information on topic *X*. It performs broad automated research: querying documents, tutorials, Q&A forums, even YouTube transcripts if available. It uses a **retrieval-augmented generation** approach to keep this grounded – for example, it will search and retrieve relevant text snippets, and incorporate them into its context for understanding [3]. This ensures factual accuracy and access to domain-specific knowledge beyond its base model's training. All sources are saved with citations in its memory.
2. **Distillation Phase (Contextual Skill Synthesis):** Everabot distills the raw information into a concise **"playbook"** for the new skill. Using the ACE framework [4], it generates a structured set of **atomic tactics or rules-of-thumb** related to *X*, then **reflects** on them and **curates** the best ones. The result is a set of **bullet-point strategies** or instructions stored in its evolving context memory. For example, if learning "problem solving," it might distill heuristics ("break problems into subproblems", "draw a diagram", etc.) each with notes, instead of just a long opaque prompt. This mitigates context collapse and brevity bias by incrementally expanding context with validated insights rather than overwriting it [4].
3. **Practice & Testing Phase:** With provisional knowledge in place, the agent **practices the new skill** and objectively evaluates itself. It will run domain-specific tests or challenges: e.g. if learning math problem-solving, it attempts a variety of math puzzles and checks results against known answers; if learning web development, it might generate a simple website and run it to see if it works; if learning a coding language, it runs unit tests or known benchmarks. It uses any available evaluation sets (for instance, HumanEval or LeetCode problems for coding, or GSM8K for math) to measure its performance. This provides a **score or success rate** for the new skill. Importantly, it also re-runs the full regression test suite for **all previously learned skills** to ensure none have fallen.
4. **Adaptive Model Update (Optional):** By design, Everabot tries to improve without modifying its base model's weights (following Reflexion and ACE principles) – instead relying on the dynamic context to store learned knowledge [5] [6]. However, if during practice it identifies that the base model's limitations are holding it back (e.g. a certain type of problem is consistently failing due to the model's inherent weaknesses), it can initiate a weight update cycle:
5. It may perform **transfer learning** on a local foundation model (e.g. fine-tune a HuggingFace LLM on a small curated dataset relevant to *X*). This fine-tuning could use lightweight methods like LoRA adapters to stay efficient. For example, after learning from text, it could fine-tune on Q&A pairs or code it wrote, to solidify the skill in the model's weights.

6. Alternatively, if fine-tuning the large model is deemed *wasteful or insufficient*, Everabot can **create a new specialist model** for the task. For instance, it might train a small neural network from scratch for a specific problem type (classification, image recognition, etc.), or spin up a new instance of a smaller LLM specialized via fine-tuning for the domain. This is essentially the agent **building its own sub-model with its own weights and biases** to handle that subset of tasks. Such a model would be integrated as a tool (called from the main agent when needed).
7. The decision to update weights vs. rely on context is made by comparing effort vs. benefit. If a skill can be reliably handled via improved prompting or code without retraining, Everabot avoids weight updates (to preserve stability). But if the skill requires fundamental capability beyond the base model (e.g. extensive new knowledge or precision), it performs a focused training. This hybrid strategy ensures **persistent improvements** when necessary, while still leaning on low-overhead context evolution for most learning.
8. **Promotion (Integration) Phase:** After practice, the agent compiles all successful strategies, any new code, and the test results. If it achieved the learning goals (e.g. >90% tests passed for the new skill) **and** all prior regressions are green, it "promotes" the new knowledge into its permanent skill library. This involves updating its long-term memory (the context datastore and any model weights if updated) and taking a **snapshot** of the entire agent state. The snapshot captures the diff of changes (see **Memory and Snapshots** below) so that progress is versioned. If the new skill did *not* meet the success criteria, or it caused a regression in older tests, the agent will **abort and revert** to the last snapshot (with a log explaining the failure). This rigorous gating ensures the agent's competence **monotonically increases or stays level** – never degrading over time.

Through these steps, if you say **"Learn game development"**, Everabot would autonomously read game programming tutorials and docs, distill key principles (game loop, rendering, physics, etc.), then perhaps build a simple game (e.g. a Pong clone) to practice, evaluate it (does it run at target framerate? any errors?), possibly fine-tune a code-generation model for Unity scripts if needed, and finally log a summary and preserve the acquired game-dev skill for future use. All of this occurs without you explicitly coding or providing data – the agent **expands its own capabilities** using online knowledge and self-driven experimentation.

Autonomous Self-Improvement Mode

In addition to learning user-specified topics, Everabot can **improve itself autonomously** when instructed (or during idle times) – this fulfills the "goes looking for ways to get better" aspect. In **Self-Improve mode**, the agent treats *its own system* as the project. It conducts an internal audit to identify weaknesses or opportunities: perhaps it notices memory usage creeping up, or a category of tasks it hasn't mastered, or new techniques in AI research it hasn't adopted. It then formulates a plan to address these. Some examples of self-directed improvements:

- **Expanding Knowledge Domains:** The agent can decide to broaden its education. For instance, after mastering web dev and problem-solving, it might target a new area like **cybersecurity** (to better protect its operations) or **mathematics** (to improve reasoning). It will then initiate a "Learn X" loop for that domain, just as if the user requested it. This way, it **curates its own curriculum** – continually adding skills that make it more general and powerful, without forgetting old ones. This open-ended learning approach is akin to an agent developing competences like Voyager's skill acquisition in an open world [7]. It will pursue topics until diminishing returns or until stopped by the user.

- **Resource Optimization:** Everabot monitors feedback like "too slow" or "using too much RAM" (it can even accept a user's complaint as a prompt to improve). Upon detecting performance issues, it enters a performance-tuning cycle (detailed in the next section). Likewise for storage: if its memory footprint grows large, it will invest effort in compressing or pruning redundant context (within the limits of not forgetting anything important). The agent essentially **treats inefficiency as a bug and fixes it** proactively.
- **Self-Maintenance & Refactoring:** The agent can refactor its own codebase (`bot/` and `app/` directories) to be cleaner, more modular, or to integrate new frameworks. For example, if a better planning algorithm or memory management pattern is known, it will rewrite its code to implement that, then run all tests to ensure behavior remains correct. If you command "improve your UI/UX," the agent might generate a local web or GUI application (say a small Flask or Electron app in `app/`) that allows easier user interaction, then integrate it into its Docker container so you have a nicer interface. Essentially, it can **rewrite any part of itself** as long as tests confirm the new version still respects all rules and retains capabilities. Over time, the codebase evolves – possibly even porting itself from Python to faster languages piece by piece.
- **Curriculum Generation (Curiosity):** The agent tries to anticipate what *unknown unknowns* might be valuable. Inspired by curiosity-driven learning, it might seek out challenges to test itself. For instance, it could scan forums or literature for tasks that current AI agents struggle with, then attempt to learn those. It might also periodically re-read the latest research (via arXiv RSS feeds) to see if new techniques (new optimization algorithms, memory architectures, etc.) could be adopted. This way, Everabot **stays up-to-date** with state of the art improvements, continuously integrating external advancements into its own design [8] [9].

In self-improvement, the same rigorous testing and rollback rules apply. The agent must **prove each change** made things better (faster, smarter, or more capable) without side effects. By chaining many small self-improvements, Everabot aims to **approach an open-ended learning system** with minimal human oversight – a step toward an evolving AGI scaffold.

Performance and Efficiency Optimization

One critical dimension of self-evolution is **speed and memory optimization**. Everabot is designed to profile its own performance and **rewrite parts of its code in lower-level languages (C/C++/Rust/Go)** when necessary for efficiency. The architecture includes a module (in `bot/rewriter/`) dedicated to this:

- **Profiling Hotspots:** The agent continuously monitors which operations consume the most CPU or memory. It uses tools like **py-spy** for Python sampling profiling (to see which Python functions use time) [10], as well as platform-specific profilers (e.g. Linux `perf` or Go's built-in `pprof` for any Go components). These tools allow Everabot to pinpoint bottlenecks without pausing the program, even in production [10]. When the user says "you're too slow" or its own metrics indicate high latency, the agent will generate a profile report and identify the top offending routines.
- **Deciding on Native Rewrites:** Once a hotspot is identified (say a Python loop doing heavy computation), the agent evaluates if rewriting that component in a faster language would yield a significant benefit. It considers the expected speedup vs. integration complexity. If the calculation is favorable (e.g. a chunk of code could run 50× faster in C++ and it's a well-contained module), Everabot will proceed to rewrite. It leverages several strategies:
- For Python code, use **pybind11** (C++) or **PyO3** (Rust) to write the critical section in C++ or Rust and expose it as a Python extension module [11] [12]. Pybind11 is a header-only library that lets C++ code

interface with Python easily [11], and PyO3 similarly allows writing Python modules in Rust with minimal friction [12]. The agent will translate the Python logic to C++/Rust, ensure it produces identical results (it writes tests for the new module to match outputs with the Python version), and then replace the Python implementation with the compiled extension. This could be used, for example, to speed up a math-intensive routine or a loop processing large data.

- In cases where concurrency or system-level performance is needed, Everabot might employ **Go**. It can spawn a Go microservice or use cgo to call C from Go and vice versa [13]. For instance, if it's doing a lot of networking or I/O bound tasks, Go's goroutines and efficient networking stack could help. The agent can compile a Go helper program and have Python communicate with it (e.g. via gRPC or simply subprocess calls), or build it as a shared library callable from Python using cgo bridging [13].
- Whichever language is chosen, the agent uses stable interface boundaries – e.g., define a clear API for the module – so that the rest of Everabot remains in Python (for flexibility), and only that module is swapped out. This modular approach ensures that even if the native rewrite fails, it can fall back to the Python version.
- **Verification and Promotion:** After rewriting a component in a faster language, Everabot runs all relevant tests to ensure the new implementation is correct (functional parity with the old one) and indeed faster. It measures runtime and memory usage of the new code against the old. If performance improves and tests pass, it integrates the new module into the system, updates documentation (in README) to note the optimization, and snapshots the state. If something goes wrong (e.g. a bug in the C++ code causing errors), it **reverts to the Python version** (the system always keeps the last working code as backup) and may log a "fallback" entry. This way, the agent improves performance *safely*, without risking a permanent break.
- **Continuous Monitoring:** Even after a successful native integration, the agent keeps monitoring. It may identify further micro-optimizations (e.g. use a different algorithm with better complexity) or realize that even the new code has hotspots (profile inside the C++ using tools or instrumenting timing). Over time, Everabot's architecture might evolve from mostly Python to a mix of Python (for orchestration) and efficient native modules for heavy-lifting – achieving high performance akin to hand-optimized software.

This automatic performance tuning draws inspiration from how a human engineer might profile and optimize a system, but here the AI agent does it autonomously. Notably, it always weighs the **cost/benefit** – it won't rewrite trivial code for a 1% gain, but it will for a large gain that impacts user experience (like a 10× speedup, or reducing memory usage from gigabytes to megabytes). By doing so, Everabot maintains a bias toward **lightweight operation**, crucial for running on a local machine without cloud supercomputers.

Memory, Knowledge Storage, and Snapshots

To support lifelong learning without running out of context window or storage, Everabot employs a sophisticated memory system:

- **Long-Term Memory Store:** Learned information (documents, strategies, code snippets) is not kept in the prompt indefinitely, which would become unsustainable. Instead, the agent uses an **external knowledge base** (likely a **vector database or a Parquet file store**). For textual knowledge, it stores embeddings of content so it can retrieve relevant pieces on the fly (that's the RAG mechanism). For structured data or test results, it uses a columnar format (Apache **Parquet**) to store tables of facts, examples, and metrics efficiently [14]. Parquet's column-oriented compression lets even large

datasets be stored compactly and read quickly, selecting only needed columns [14]. This is important as Everabot's knowledge grows – it might accumulate thousands of Q&A pairs, code templates, etc., so using a compact, searchable format prevents memory blowup.

- **Prompt Compression:** When assembling the prompt/context that goes into the LLM at each step, Everabot applies compression techniques to keep it concise. For example, it might summarize older context (since it has the distilled bullet points, those are already concise) or use specialized algorithms like **LLMLingua** which can compress lengthy prompts up to 20× while preserving semantics [2]. This allows the agent to utilize **very long context windows** (if the model supports, say, 16k or 100k tokens) effectively by packing more information into fewer tokens. The agent's reflection mechanism also works to remove or archive details that are no longer needed in live context (e.g. once a piece of knowledge is codified into a skill or a test is passed consistently, the raw reference can be stored in long-term memory and dropped from the active prompt). These steps avoid *context overflow* and **reduce inference costs** for the LLM.
- **State Snapshots (Delta Checkpoints):** Each time the agent's state is promoted (after learning or self-improvement), it creates a **snapshot** in the `snapshots/` directory. However, storing a full copy of the model and data each time would be huge. Instead, Everabot uses **delta encoding** for snapshots. It keeps a base state and for each new snapshot, it stores only the **difference (diff)** – likely using the VCDIFF format (RFC 3284) [15] or similar. VCDIFF is a generic differencing format that encodes changes compactly and portably [15]. For example, if a new skill added 100KB of data and modified 3 files, the snapshot might just be those changes encoded in a few KB. This allows **frequent checkpoints** without much storage cost. The delta approach also means rolling back is trivial – the agent can apply the reverse of a delta to go to a previous state if needed. All snapshots are timestamped and versioned, so a complete history of the agent's evolution is retained (much like a git repository of itself).
- **No Old Skill Left Behind:** To prevent forgetting, Everabot sometimes **refreshes** old knowledge. If a skill hasn't been used in a while, it might re-run a quick test or revisit key points from memory to ensure they're still valid. If an update affected some old data structure or API, it can detect discrepancies when running regression tests and then fix or update the old skill accordingly. This is akin to **periodic memory rehearsal** to combat drift. Because the detailed context for each skill is stored in long-term memory (and not always in the active prompt), this retrieval and rehearsal is important: it can pull up the archived context for an old skill when needed and re-inject it to solve a problem or answer a question in that domain. In doing so, it may revise that context if new knowledge supersedes it, always aiming to keep the knowledge base consistent and cumulative.

In summary, Everabot's memory architecture ensures it can **scale its knowledge without running out of capacity**. The combination of an external knowledge base, compression, and diff-based snapshots provides a backbone for potentially years of continuous learning. It's inspired by how humans maintain long-term knowledge (external notes, periodic review) rather than trying to keep everything in immediate memory.

Adaptive Model Evolution (Foundation Model and Custom Models)

While much of Everabot's improvement relies on context engineering and external tools, the foundation LLM itself can be improved over time:

- **Foundation Model Setup:** Initially, Everabot uses a local Large Language Model (LLM) as its core (for example, a smaller open-source model that can run on a PC – perhaps a 7B or 13B parameter model loaded via HuggingFace). This base model is essentially the "brain" for reasoning and code

generation. It starts off pre-trained on general data (no specialized knowledge of the user's tasks yet).

- **Secure Base and Backups:** The original weights of the foundation model are kept as a secure backup. If the agent fine-tunes or alters a model and things go awry, it can always fall back to the original or an earlier stable version of the model. This is important: any time we do weight updates, we **branch the model checkpoint** rather than overwriting it, until it's proven stable. The system might maintain multiple model versions (the current fine-tuned one and the base) and even use the base as a "sanity checker" (e.g. to compare answers or to help debug when the fine-tuned model fails).
- **Lightweight Fine-Tuning:** When needed, Everabot can perform fine-tuning on the base LLM with new data it generated or found. For instance, after learning a domain, it might compile a Q&A dataset from its knowledge and fine-tune the model on it to improve accuracy on that domain. This is done carefully to avoid overfitting or degradation of other capabilities. Techniques like **LoRA (Low-Rank Adapters)** or other parameter-efficient fine-tuning are preferred to adjust weights without a full costly update. This allows *local training* even on a modest machine. The agent monitors evaluation metrics on a validation set (including general tasks) during fine-tuning – it will stop or rollback if it detects any sign of the model forgetting general capabilities. The idea is to **extend the model's proficiency** incrementally, not radically change its core.
- **Building Specialized Models:** In scenarios where a task might be better served by a different architecture, Everabot can design a small model itself. For example, if asked to "learn to read handwritten digits," it might decide to train a simple convolutional neural network (CNN) using an existing dataset (like MNIST) it downloads, instead of relying on the LLM for vision. The agent has the capability to write and run training code (using frameworks like PyTorch or TensorFlow) for such purposes. These specialist models (say `digit_recognizer.pth` in `bot/` or in a `models/` subfolder) are then used by the agent as needed (maybe via a Python binding). Essentially, Everabot can act as an AutoML system for sub-tasks: designing model architecture, training it on data, and deploying it – all autonomously. Each such model is subjected to tests (e.g. accuracy on test data) before use.
- **Ensemble and Arbitration:** After multiple fine-tunings or new models, Everabot could employ an ensemble strategy: using the right model for the right context. For instance, use the specialized math-tuned LLM for math problems, but use the base model for general conversation, etc. It keeps track of which model performs best on which kind of task (via its testing logs). An arbiter module can route queries to the appropriate expert model. This way, the system's overall performance keeps improving as it accumulates expert subsystems, rather than trying to force one monolithic model to handle everything.
- **Model Update Example:** Suppose Everabot's base LLM initially struggles with coding in Rust language. After learning Rust through context, it still sees inconsistent results. The agent might generate a fine-tuning dataset by solving many Rust programming tasks (using external compilers to verify solutions) and then fine-tune the model on that. The result would be a model more proficient in Rust code generation. It would then validate that this fine-tuned model **still performs well on other tasks** like Python coding or general Q&A. If successful (perhaps even seeing *improved* general performance due to broader training feedback), it adopts the new weights. If it noticed any drop (e.g. natural language answers got worse), it could decide to **keep a separate Rust-specialized model** and not alter the original for general tasks. This nuanced approach ensures *robust overall capability* grows without unintended side-effects.

In essence, Everabot's design doesn't treat the model's weights as static. They are another aspect of the system that can evolve – albeit under tight control. By combining **Agentic Context Engineering (which**

proved you can adapt without weight updates) [5] with **occasional weight optimization**, Everabot can potentially exceed the limitations of either method alone. Notably, the ACE research showed that even a smaller model with evolving contexts can match a larger fixed agent [16], so Everabot leverages that first – but it also has the freedom to *grow new "brain cells"* (train new models) if that brings an advantage.

Example User Flows

To illustrate how Everabot functions in practice, here are a few example scenarios and how the agent would handle them:

- **User says "Learn problem solving."**
  Agent's process: It interprets this as a broad skill acquisition in analytical problem-solving (perhaps focusing on puzzles or logical reasoning). The agent researches classic problem-solving techniques (searching for terms like "problem solving strategies", "heuristics in AI"). It finds sources like academic papers or forums discussing approaches. It compiles a list: e.g. *"Understand the problem, devise a plan, carry out the plan, review"* (Polya's steps), or heuristics like *"solve a simpler analogous problem first"*. It adds these as bullets to its context memory [4]. Then it goes to practice: it might pull sample problems (math puzzles, logic riddles) from the web. For each, it tries to apply its strategies and uses the LLM to propose solutions, then checks if the answers are correct (many puzzle sources have answers available, or it can simulate execution for logic puzzles). Suppose it gets 8 out of 10 puzzles right initially. It identifies where it went wrong (say it misunderstood a trick in a riddle). It refines its strategy bullet points (adding, for example, *"double-check for hidden assumptions in puzzle wording"*). It practices more until it consistently solves, say, 95% of a test set of puzzles. Meanwhile, it ensures its prior abilities (e.g. coding, general Q&A) remain fine – it reruns those tests, and they pass. It then finalizes learning: logs its strategy notes in a human-readable form in the `snapshots/` log, and maybe fine-tunes a bit if it created a new dataset of Q&A. **Outcome:** The user receives a report that Everabot has learned problem-solving, listing strategies and showing before/after test results, and now the agent can tackle new problems effectively on request.

- **User says "Build me a basic website."**
  Agent's process: Now Everabot is tasked with a concrete project deliverable. Let's assume it has not built websites before. It will treat this as a mission that might involve learning new skills (web development) on the fly. First, it scopes the task: a "basic website" likely means HTML/CSS/JS or maybe a simple web app. It decides on a tech stack (perhaps a static site or a tiny Flask application, depending on context). It checks its knowledge – maybe it doesn't know much about HTML. So it enters a mini "Learn web dev" loop: reads some HTML/CSS tutorials quickly, distills key tags and styles. It won't aim for mastery, just enough to proceed. Then it generates the code for the website (using the LLM to output HTML/CSS files in the `app/` or a new `website/` directory). It might open a local browser (via a tool or headless mode) to verify the page looks as intended. If the user provided specifics (like "a website for my portfolio"), it customizes content accordingly. After building, it tests the site: ensures links work, it's responsive, etc. It then uses GitHub's API (since the user mentioned "in my account") – Everabot can use an access token (provided securely) to **push the code to a GitHub repository** under the user's account. It creates commit messages documenting what it did. Throughout, it logs its actions. If any part fails (e.g. GitHub push issues), it consults error messages and fixes them. Also, since web dev is new, after completing the mission, Everabot adds the newly learned web dev techniques to its skill library for future reuse. **Outcome:** The user gets a link to a GitHub repo with a working basic website and a summary of how the agent built it (e.g.

"Created a responsive HTML5 page with CSS, validated with W3C checker, and pushed to repo *username/BasicSite*"). The agent also doesn't forget this – next time, it will do web tasks even faster.

- **User says "Make a game for me, and keep updating it with new features."**
  Agent's process: This is an open-ended creative project. Everabot would break it down: first learn any needed game development skills (if not already known). Perhaps it decides to use a simple game engine or library (like Pygame for Python, or Unity if it's ambitious and has the environment). Let's say it uses Pygame for a simple 2D game. It researches Pygame docs and tutorials, distills basics (game loop, event handling, drawing), and maybe fine-tunes its code generator on example game snippets. Then it creates an initial version of the game – maybe a basic platformer or puzzle game as requested by user. It runs it to test playability (since it can execute code, it could simulate key presses or just ensure no crashes). The user also asked for continuous updates with new features. Everabot can then enter a loop where it periodically asks itself (or the user) for new feature ideas: e.g. "Add a scoring system," "Add enemy characters," etc. For each feature, it modifies the code, tests the game again, and ensures previous features still work (no regression – it has automated game tests, maybe using image snapshots or logs). Over time, the game grows in complexity under the agent's autonomous development. If the game's performance drops (more enemies = slower), the agent might optimize the code (e.g. use Cython or move computation to C++). If the scope grows significantly, the agent might even refactor the structure (modularize code, use design patterns from game programming). It also keeps an eye on not breaking any rules (if there's a rule like "no graphic violence," it won't suddenly add extreme content). **Outcome:** The user observes their game evolving version by version (the agent might provide them new builds or push updates to a repo). The logs show each feature added and how it was implemented. Essentially, Everabot acts as an autonomous software developer, from initial commit to iterative improvements – similar to an AutoGPT but with persistent learning. The user can intervene with suggestions, or let it continue until it reaches a satisfactory state.

- **User says "Improve your own UI and UX."**
  Agent's process: This triggers Self-Improve mode focusing on the interface. Suppose currently the user interacts via a command-line or chat interface. The agent decides this could be improved with a graphical UI. It considers options: maybe a simple local web dashboard using Flask or a desktop GUI using something like Tkinter or Electron. Let's say it chooses a web dashboard for flexibility. It **refactors its** `app/` **code** to include a small web server that can accept user commands and display logs nicely. It writes HTML/JS for the frontend (maybe even uses a small JS framework if it knows one, or keeps it simple). It then launches this UI and tests that it can issue commands to itself and get responses. It ensures all original CLI functionality still works (regression test: the old interface paths are intact for backward compatibility). If the user had set any constraints (like "do not expose certain features on the UI"), it respects those. Once done, it prompts the user to open the interface on a local port to try it out. **Outcome:** The agent has **self-upgraded its user experience**, making it easier to use. It likely writes in the README how to use the new UI. If any issues are found (perhaps the user doesn't like the design), the agent can iterate further. This demonstrates the reflexive capability: improving itself in ways beyond just "learning facts" – it can alter the very way it interacts with the user for the better.

- **Agent "Learn to read SVG/PNG/DXF files" (internal goal or user command).**
  Agent's process: This example (inspired by the provided context) shows how Everabot would acquire a very specific technical skill – parsing new file formats until 100% accuracy. Let's use **SVG (Scalable**

Vector Graphics) as the target. The agent would obtain the SVG specification (from W3C) [17] and any related documentation, plus sample SVG files. It studies the spec to understand the format structure (XML-based vector graphics description). It then proceeds to implement a parser: likely writing code in Python to parse an SVG XML into some internal representation (or maybe using an existing library, but the exercise implies it should "learn to read" meaning possibly implement from scratch to prove understanding). It uses incremental development: parse basic shapes first, then gradually handle more features. After each update, it tests on sample SVG files: can it successfully parse them without error and maybe even render them (if a rendering test is possible)? It keeps track of coverage: e.g., if out of 100 sample SVGs, initially 60 parse, 40 fail on unsupported features. It then implements those features and so on, until all 100 parse (near 100% coverage). It also references the spec to ensure conformance (like it might validate that its output matches expected values from known reference renderings or that it respects all required tags/attributes). It would do similarly for **PNG** (reading the binary format per the PNG spec RFC 2083 [18], handling chunks like IHDR, IDAT, etc.), or CAD formats like DXF/DGN (which are more complex binary formats – the agent might utilize some open specs or libraries as references). Through this, Everabot essentially **teaches itself new file format knowledge** to the point of implementation competence. **Outcome:** The user (or the agent's own system) gains the ability to read those file types. This could then feed into other tasks (e.g. if later asked to convert an SVG to PNG, it now has the parser and could implement a basic renderer). The key achievement is the agent's demonstrated *complete learning cycle*: from spec to full implementation with tests confirming success.

These scenarios highlight Everabot's versatility: it can function as a researcher, a developer, a project manager, or a self-tuner as needed. In each case, the interplay of its components – research module, skill library, test harness, code generator, performance tuner – and the strict validation regime ensure a reliable outcome.

Architecture and File Structure

Everabot's implementation is organized into a clear modular structure. Below is the proposed project layout (folders and key files) and an explanation of each part:

```
Everabot/
├── ruleset/            # Core rule definitions and behavior profiles
├── test/               # Test suites for skills and regression checks
├── app/                # User interface layer (CLI, GUI, or API endpoints)
├── bot/                # The agent's brain: logic for learning, planning,
memory, etc.
├── snapshots/          # Stored snapshots of agent state (knowledge & code
diffs)
├── Dockerfile          # Container setup for running Everabot locally
├── pyproject.toml      # Python project configuration (using Poetry for deps)
├── poetry.lock         # Locked dependency versions for reproducibility
└── README.md           # Documentation and usage instructions
```

Let's break down the components:

- `ruleset/`**:** This directory contains the **definition of Everabot's rules and personas**. It includes files that specify the **global behavior constraints** (for example, a YAML or JSON file of rules: "Do not produce disallowed content", "Always preserve prior skills unless explicitly replaced", etc.). It may also contain **profiles for different domains** – for instance, a configuration for "game_dev" that outlines what success means in that context, or a profile for "self_improvement" that lists safe operations. Essentially, `ruleset/` is where the developer (or user) can update what the agent is allowed to do and how it should score success. On initialization, Everabot loads these rules into memory and they govern its planning and decision-making. By externalizing rules here, one can easily update the agent's constraints without altering code.
- `test/`**:** This directory holds all automated tests that Everabot uses to validate performance. It is further structured by domain or component. For example, `test/math/` might contain math problem sets and an evaluator script; `test/code/` might have coding challenges or unit tests for generated code; `test/perf/` might include performance benchmarks (scripts to measure memory usage, speed of certain functions). Importantly, this folder includes the **baseline (turning) tests** for the base LLM's capabilities – essentially a sanity check suite that the original model can pass (like simple arithmetic, basic reasoning, etc.). These baseline tests are run after every change to ensure the agent didn't lose core functionality. As Everabot learns new skills, new tests are added here. For instance, after learning web dev, a test might be "build a minimal HTML page and verify output." The agent can also generate tests for itself and place them here. This folder acts as the **quality gatekeeper** for the project.
- `app/`**:** This is the user interaction layer. Initially, it might just contain a simple CLI interface (maybe a `main.py` that reads user input from console and feeds it to the `bot`). Over time, as the agent improves its UI, this folder could include a web server (`server.py`), HTML/JS files for a web UI, or other frontends (maybe a Qt GUI). The separation of `app` ensures the core logic is decoupled from how users interact with it. You could have multiple frontends (one for command-line, one for a GUI) all calling into the same `bot` logic. For example, `app/cli.py` could parse command-line arguments and then invoke `bot/agent.py` methods. If Everabot implements an API, it might live here as well (for instance, a FastAPI app that exposes endpoints to issue "learn" or "build" commands remotely). The `app` layer also handles any output formatting for the user – e.g., printing pretty logs or hosting a dashboard.
- `bot/`**:** This is the heart of Everabot – all the intelligent behaviors reside here. It's likely further subdivided into modules:
- `bot/agent.py`: the main class orchestrating high-level behavior (interpreting commands, switching between Mission and Self-Improve modes, managing the loop of generate-reflect-curate).
- `bot/memory.py`: manages the memory store (interfaces with the vector database or Parquet files in `snapshots/`), handles retrieval and saving of context.
- `bot/planner.py`: logic for deciding what step to do next (e.g. given a goal, break into subgoals, or given a performance issue, decide to profile).
- `bot/learner.py`: implementations of the learning pipeline (the RAG research functions, the distillation function, practice/test execution).
- `bot/rewriter/`: a subpackage for code optimization – e.g. `rewriter/profile.py` to run profilers, `rewriter/translate.py` to assist in code translation to other languages, `rewriter/compile.sh` scripts if needed.
- `bot/evaluator.py`: code for running tests from the `test/` directory and gathering results.

- `bot/models.py`: helpers for loading different model versions (base model, fine-tuned models, any custom smaller models).
- possibly `bot/webclient.py`: if the agent needs to fetch data from internet (with proper safeguards).

In essence, `bot/` contains the **algorithmic brain** and all subroutines for learning and self-editing. It is pure logic and does not do any user I/O directly (that's `app`'s job). By reading this code, one should understand how Everabot thinks, plans, learns, and self-monitors. This is the part that would reference research like ACE, Reflexion, etc., in code comments and implement those ideas (for example, a `reflect()` method where the agent critiques its last answer, inspired by Reflexion [5]).

- `snapshots/`**:** This folder stores the agent's state checkpoints. Each snapshot could be a folder or archive containing: a diff file (perhaps `.vcdiff`) representing changes from the previous snapshot, any additional artifacts (like newly fine-tuned model weights or new test files), and a log of what changed. The naming might be timestamp-based or incremental (snapshot1, snapshot2, … or dates). There might also be a "current" symlink pointing to the latest. On startup, Everabot can load the latest snapshot to resume where it left off. The use of delta encoding means inside a snapshot file you'd find something like "diff from previous: +Added rules in ruleset/web.yaml, ~Modified 20 lines in agent.py, +New file bot/models/web_model.pth (some binary data)". The system could provide tools to merge these if needed or inspect them. Additionally, `snapshots/` might contain the vector memory store (if using Parquet or a SQLite DB for knowledge, it could be here or in a subfolder). By keeping all this versioned, one can analyze the agent's learning trajectory or revert if a bad update slipped through (though the design tries to prevent that).
- `Dockerfile`**:** A recipe to build a Docker image with all necessary dependencies (Python, compilers for C++/Rust/Go, any libraries like PyTorch or TensorFlow if training models, etc.). Running Everabot in Docker ensures consistency across environments – critical for a complex system. The Dockerfile would likely start from a base (maybe `python:3.10-slim`), install system packages for compilers (e.g. `build-essential`, Rust toolchain, Go if needed), then copy the project code, install Python dependencies (via Poetry, which uses `pyproject.toml` and `poetry.lock` to get exact versions), and set up an entrypoint to launch the `app`. This allows someone to **spin up Everabot with a single command** and also limits it (for safety, Docker can sandbox what it can access – though if internet access is desired for research, the container needs net access). It's also useful for resetting the agent if needed or running multiple instances.
- `pyproject.toml` **and** `poetry.lock`**:** These define the Python environment. `pyproject.toml` will list dependencies such as: Transformers (for the LLM), databases (maybe `faiss` or `chromadb` for vector store, or `pyarrow` for Parquet), profiling tools (`py-spy`), etc., as well as the project's own package structure. It may have dev dependencies like testing frameworks (pytest). The `poetry.lock` locks them to specific versions to avoid unpredictability. This ensures that if someone else (or the agent itself on another machine) sets up the project, they get the same versions that have been tested – important for reproducibility of learning outcomes.
- `README.md`**:** The top-level documentation. It explains what Everabot is, how to install/run it, and provides examples of usage. As the agent evolves, it might update this README to document new features or changed commands. (Yes, the agent could edit its own README when it adds a new ability – providing transparency to the user). The README would also likely include a **quickstart guide** ("Run `docker build .` then `docker run everabot` to start, then type `learn "X"`") and possibly an architecture overview (some of which would be similar to this whitepaper's content). This file is crucial for new users to understand how to interface with Everabot. Over time, it might also accumulate notes or links to the research inspirations behind it (since the user wanted a whitepaper style, some of those references might live here for completeness, albeit in a more condensed form).

Overall, this file structure enforces a separation of concerns: **policy vs. mechanism**, **interface vs. core logic**, **knowledge vs. code**, **history vs. current state**. Such organization not only makes the system easier to navigate and maintain, but also aligns with the agent's own operation (for example, rules are clearly separated from code, so the agent can update code without accidentally changing the fundamental rules).

Feasibility and Next Steps

Building Everabot is an ambitious but **feasible** project, grounded in current research advancements and engineering practices. The outline above draws from *state-of-the-art techniques*: evolving prompt strategies instead of constant retraining [1], self-reflection for error correction [5], open-ended skill acquisition like Voyager's skill library [7], and automated coding agents as seen in SWE-Agent [19]. Each individual component has precedent:

- *Agentic Context Engineering (ACE)* has demonstrated that an LLM can improve via context updates, achieving significant gains with a smaller model by accumulating knowledge in prompts [9]. Everabot's design builds on that, ensuring context evolution is done in a structured, non-collapsing way.
- *Reflexion* and similar frameworks show the power of an agent writing **verbal feedback to itself** instead of fine-tuning weights [5]. Everabot uses this to handle trial-and-error learning efficiently – it's essentially reinforcement learning via natural language critique instead of gradient descent.
- *Automated coding* abilities are evidenced by systems like SWE-Agent, which allowed an AI to navigate and modify a codebase and solve software tasks [19]. Everabot takes this further by not just editing a static codebase, but by **continuously rewriting and generating its own codebase**, including its tests and documentation.
- Tools for efficient storage (Parquet, VCDIFF) and performance (py-spy, pybind11, etc.) are mature and available. The integration of these is engineering work, not fundamental research – meaning we can implement them with some effort and the agent can learn to use them.

In practical terms, to get started one would: 1. Set up the base environment (choose a suitable open-source LLM that can run locally, e.g., a 7B parameter model for initial tests, and containerize it). 2. Implement the core loop (perhaps start with a simpler version of the learning loop, and a simple memory store). 3. Incrementally add components: e.g. first get it to do a basic "learn from one document and recall info" with RAG, then add testing harness, then allow it to write to its own code. Each step can be tested in isolation (for instance, ensure that it can indeed modify a function and not break tests – a bit like how Github's Copilot may suggest code changes, but here the agent does it autonomously and tests). 4. Gradually introduce the more advanced behaviors (profiling, spawning new model training, etc.) as the need arises and as the simpler versions prove stable.

Throughout development, keeping the **explainability and logging** is key – this not only helps the user trust the agent, but will help the developer debug the agent's thought process (since you can see in logs what it *thought* and where it might have gone wrong).

In conclusion, Everabot's architecture merges ideas from the latest research with a robust engineering framework for continuous integration of skills. By following this outline – adhering to strict validation, maintaining modularity, and leveraging both context and learned parameters – one can build a system that **autonomously grows smarter and more capable over time**. This endeavor is complex, but the outline

and references here provide a roadmap. If successful, the outcome is a versatile AI assistant that **never stops learning**, ultimately handling an ever-expanding array of tasks as a true jack-of-all-trades AI.

References (Key Technical Sources)

1. **Agentic Context Engineering (ACE)** – Q. Zhang *et al.*, *"Evolving Contexts for Self-Improving Language Models."* arXiv:2510.04618 (Oct 2025). Describes improving LLMs via evolving prompt strategies instead of weight updates [1] [9].
2. **LLMLingua (Prompt Compression)** – H. Jiang *et al.*, *"LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models."* EMNLP 2023. Presents methods to compress prompts up to 20× with minimal loss [2].
3. **Reflexion (Self-Reflection for Agents)** – N. Shinn *et al.*, *"Reflexion: Language Agents with Verbal Reinforcement Learning."* arXiv:2303.11366 (Mar 2023). Introduces agents that improve via linguistic self-feedback, without updating weights [5] [6].
4. **Apache Parquet (Columnar Storage)** – *Parquet Documentation*, Apache (2025). Parquet is an open-source column-oriented data file format with efficient compression and encoding, ideal for large-scale data storage [14].
5. **Retrieval-Augmented Generation (RAG)** – P. Lewis *et al.*, *"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks."* NeurIPS 2020. Demonstrates augmenting LLMs with external document retrieval to improve factual accuracy [3].
6. **Py-spy Profiler** – B. Frederickson, *py-spy: Sampling profiler for Python programs*, GitHub (2018–2025). A low-overhead profiler that can visualize Python program execution without code changes [10]. Useful for identifying performance bottlenecks in Everabot.
7. **pybind11 (C++ Bindings for Python)** – W. Molden *et al.*, *pybind11 v2.10 documentation*. A lightweight header-only library that exposes C++ types/functions to Python, enabling seamless creation of Python C++ extensions [11]. Allows Everabot to integrate optimized C++ code.
8. **PyO3 (Rust Bindings for Python)** – PyO3 Development Team, *PyO3 User Guide v0.27* (2024). Enables writing Python modules in Rust and embedding Python in Rust programs [12]. Everabot can use this to incorporate Rust for performance-critical components.
9. **cgo (Go Interoperability)** – *Go Wiki: cgo*, golang.org (2023). Documentation on calling C/C++ from Go and vice versa [13]. Provides Everabot a pathway to use Go's capabilities by interfacing with C, useful for concurrency or system-level tasks.
10. **RFC 3284 – VCDIFF (Delta Encoding)** – D. Korn *et al.*, IETF (June 2002). Defines the VCDIFF format for generic differencing and compression of data [15]. Everabot uses this to store snapshot diffs efficiently.
11. **PNG Specification** – T. Boutell *et al.*, *PNG (Portable Network Graphics) Specification Version 1.0*, RFC 2083 (1997). Image format spec referenced when Everabot learns to parse PNG files [18].
12. **SVG Specification** – W3C SVG Working Group, *Scalable Vector Graphics (SVG) 2*, W3C Candidate Recommendation (Oct 2018). Vector graphic format spec for when Everabot learns SVG parsing [17].
13. **Voyager (Lifelong Learning Agent)** – H. Wang *et al.*, *"Voyager: An Open-Ended Embodied Agent with Large Language Models."* arXiv:2305.16291 (May 2023). Demonstrates an LLM-driven Minecraft agent that generates its own curriculum, builds a skill library, and iteratively improves [7]. Inspires Everabot's skill library and auto-curriculum ideas.
14. **SWE-Agent (Autonomous Software Engineer)** – J. Yang *et al.*, *"SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering."* arXiv:2405.15793 (Nov 2024). Shows that an agent with a

proper interface can navigate file systems, write and modify code, and solve coding tasks autonomously [19]. Validates Everabot's approach to self-modifying its code under test guidance.

---

[1] [4] [8] [9] [16] [2510.04618] Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models
https://arxiv.org/abs/2510.04618

[2] [2310.05736] LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models
https://arxiv.org/abs/2310.05736

[3] "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" Summary | by David Min | Medium
https://medium.com/@dminhk/retrieval-augmented-generation-for-knowledge-intensive-nlp-tasks-summary-fc6dc7c4900e

[5] [6] [2303.11366] Reflexion: Language Agents with Verbal Reinforcement Learning
https://ar5iv.labs.arxiv.org/html/2303.11366

[7] [ML] Voyager: An Open-Ended Embodied Agent with Large Language Models | by Ming-Hao Hsu | Medium
https://medium.com/@amiable_cardinal_crocodile_398/ml-voyager-an-open-ended-embodied-agent-with-large-language-models-b5efe4bca1ad

[10] py-spy · PyPI
https://pypi.org/project/py-spy/0.1.0/

[11] pybind11 documentation
https://pybind11.readthedocs.io/en/stable/

[12] Introduction - PyO3 user guide
https://pyo3.rs/v0.27.0/

[13] Go Wiki: cgo - The Go Programming Language
https://go.dev/wiki/cgo

[14] Overview | Parquet
https://parquet.apache.org/docs/overview/

[15] RFC 3284: The VCDIFF Generic Differencing and Compression Data Format
https://www.rfc-editor.org/rfc/rfc3284.html

[17] Scalable Vector Graphics (SVG) 2
https://www.w3.org/TR/SVG2/

[18] Information on RFC 2083 » RFC Editor
https://www.rfc-editor.org/info/rfc2083

[19] [2405.15793] SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering
https://arxiv.org/abs/2405.15793